FIAP

Engenharia de Software

Computational Thinking with Python

Câmera Inteligente com OCR

Sprint 4 – Documentação Técnica

Integrantes do Grupo:

Artur Fabi Brandi – RM570258

Felipe Santos Ribas – RM569121

Luiz Gonzaga – RM572446

Orion Cavalcante França – RM573677

Victor Lula Heineken Rodrigues – RM570782

São Paulo, 2026

SUMÁRIO

1 Descritivo do Projeto 3

1.1 Contexto e Motivação 3

1.2 Objetivo Geral 3

1.3 Justificativa 3

1.4 Público-Alvo 3

2 Descrição das Funcionalidades Implementadas 4

2.1 Capturar Texto (OCR de imagem ou digitação) MODIFICADO 4

2.2 Pesquisar Texto no Google MODIFICADO 4

2.3 Traduzir Texto MODIFICADO 5

2.4 Saiba Mais na Wikipédia NOVO 5

2.5 Visualizar Histórico (busca e filtro) AMPLIADO 5

2.6 Editar Captura NOVO 5

2.7 Remover Captura e Lixeira NOVO 6

2.8 Ver Estatísticas AMPLIADO 6

2.9 Exportar e Importar Capturas NOVO 6

2.10 Limpar Histórico MODIFICADO 6

3 APIs Externas Consumidas NOVO 6

4 Persistência de Dados NOVO 7

5 Tratamento de Exceções NOVO 8

6 Conceitos de Python Aplicados 8

7 Estrutura do Código-Fonte 9

8 Justificativa de Alterações em Relação à Sprint 3 NOVO 11

8.1 O que foi removido 11

8.2 O que foi modificado 12

8.3 O que foi adicionado 12

8.4 Atendimento ao feedback da Sprint 3 13

1. DESCRITIVO DO PROJETO

1.1 Contexto e Motivação

O fluxo de pesquisa acadêmica envolve etapas repetitivas: localizar um texto (em livro, lousa, slide ou documento), digitá-lo manualmente em um buscador ou tradutor, e só então obter a informação desejada. Esse processo gera fricção desnecessária e dispersa o foco do estudante.

A feature proposta para a câmera JOVI elimina essas etapas ao integrar a câmera do smartphone com ferramentas de pesquisa, tradução e consulta, utilizando OCR (Reconhecimento Óptico de Caracteres) para capturar textos do ambiente e convertê-los em ações imediatas.

1.2 Objetivo Geral

Desenvolver, em Python, a implementação final do fluxo de uma câmera inteligente com OCR: reconhecimento real de texto em imagens, armazenamento persistente das capturas, pesquisa no Google, tradução automática e consulta à Wikipédia, além da gestão completa do histórico (edição, remoção, lixeira, busca e estatísticas). Na Sprint 4 o sistema deixa de ser uma simulação: o texto passa a ser lido da imagem por uma API de OCR, as traduções são feitas de fato por uma API de tradução e todos os dados são mantidos entre execuções em arquivo JSON.

1.3 Justificativa

Os cenários de uso contemplados são diretamente ligados ao cotidiano universitário:

- Fotografar o conteúdo da lousa e pesquisá-lo sem trocar de aplicativo;
- Fotografar trechos de livros em língua estrangeira e obter a tradução imediatamente, já salva junto ao texto;
- Entender rapidamente um termo desconhecido por meio de um resumo da Wikipédia; NOVO — SPRINT 4
- Corrigir erros de reconhecimento editando o texto capturado; NOVO — SPRINT 4
- Manter o histórico do semestre inteiro, organizado por categoria e consultável por palavra-chave; NOVO — SPRINT 4
- Acompanhar, pelas estatísticas, que tipo de conteúdo o estudante mais captura e em que período.

O diferencial da solução é transformar a câmera de ferramenta passiva de captura em ferramenta ativa de pesquisa e aprendizado.

1.4 Público-Alvo

Estudantes universitários em aulas presenciais ou híbridas que precisam pesquisar, traduzir ou registrar conteúdos de forma rápida durante o processo de estudo.

2. DESCRIÇÃO DAS FUNCIONALIDADES IMPLEMENTADAS

O programa apresenta um menu com 12 funcionalidades e a opção **0 (Sair)**. Após a execução de cada funcionalidade, o programa retorna ao menu principal.

```
==================================================
       CÂMERA INTELIGENTE COM OCR
       3 captura(s) salva(s)
==================================================
  1. Capturar texto (OCR de imagem ou digitação)
  2. Pesquisar texto no Google
  3. Traduzir texto
  4. Saiba mais na Wikipédia
  5. Ver histórico (com busca e filtro)
  6. Editar captura
  7. Remover captura
  8. Lixeira (restaurar / esvaziar)
  9. Estatísticas
 10. Exportar capturas (arquivo JSON)
 11. Importar capturas (arquivo JSON)
 12. Limpar histórico
  0. Sair
==================================================
```

2.1 Capturar Texto (OCR de imagem ou digitação) MODIFICADO

O usuário informa o caminho de uma imagem (ou a URL de uma imagem pública) e o idioma do texto. A imagem é enviada à API OCR.space, que devolve o texto reconhecido. O texto é exibido para confirmação e, em seguida, o usuário classifica o tipo de fonte e a categoria. Caso o OCR falhe (sem internet, imagem inválida ou nenhum texto legível), o sistema oferece a digitação manual como alternativa. Se a chave do OCR.space não estiver configurada, o sistema avisa que o OCR está indisponível e oferece diretamente a digitação manual.

Regras de negócio:

- Aceita imagens `.png`, `.jpg`, `.jpeg`, `.gif`, `.bmp`, `.tif`, `.tiff` e `.webp` de até 1 MB (limite do plano gratuito da API);
- Textos com menos de 3 caracteres úteis são rejeitados;
- Tipo de fonte (impresso, lousa/slide, manuscrito) e categoria (Livros, Programação, Estudos, Idiomas, Documentos, Outros) são obrigatórios e validados em loop;
- Cada captura recebe um `id` único, data de criação e a origem (`"ocr"` ou `"manual"`), e é salva imediatamente em arquivo.

2.2 Pesquisar Texto no Google MODIFICADO

Gera a URL de pesquisa no Google para uma captura escolhida pelo usuário e oferece abri-la no navegador.

Regras de negócio:

- Disponível apenas se houver ao menos uma captura no histórico;
- O texto é codificado via `urllib.parse.quote` para compor a URL de forma segura;

- Após exibir o link, o sistema pergunta se o usuário deseja abri-lo no navegador padrão (módulo `webbrowser`); se não for possível abrir, o link continua na tela para ser copiado.

2.3 Traduzir Texto MODIFICADO

Traduz a captura escolhida por meio da API MyMemory, com detecção automática do idioma de origem. A tradução é exibida e armazenada na própria captura, junto com o link do Google Tradutor como alternativa.

Regras de negócio:

- Idiomas de destino: Português, Inglês, Espanhol, Francês e Alemão;
- Textos acima de 500 caracteres são truncados (limite da API), com aviso ao usuário;
- É mantida apenas a tradução mais recente de cada idioma;
- Se a API falhar ou o texto já estiver no idioma de destino, uma mensagem clara é exibida e o link do Google Tradutor é oferecido (com a opção de abri-lo no navegador).

2.4 Saiba Mais na Wikipédia NOVO

Pesquisa o texto capturado na Wikipédia em português e exibe o resumo do artigo mais relevante, com o link para a página completa.

Regras de negócio:

- Utiliza a busca da Wikipédia para encontrar o artigo e o endpoint REST de resumo para obter o texto;
- O termo de busca sugerido (início do texto capturado) pode ser confirmado com Enter ou substituído por outro digitado pelo usuário;
- Se nenhum artigo for encontrado, o usuário é informado.

2.5 Visualizar Histórico (busca e filtro) AMPLIADO

Exibe as capturas com data, tipo de fonte, categoria, origem, texto completo, traduções salvas e data da última edição. O usuário pode buscar por palavra-chave e filtrar por categoria.

Regras de negócio:

- A busca não diferencia maiúsculas de minúsculas;
- Busca e filtro são opcionais e podem ser combinados; o total exibido e o total geral são informados.

2.6 Editar Captura NOVO

Permite corrigir o texto (útil para erros de reconhecimento do OCR), o tipo de fonte e a categoria de uma captura.

Regras de negócio:

- Pressionar Enter mantém o valor atual de cada campo;
- A data de edição é registrada; se o texto mudar, as traduções antigas são descartadas, pois deixam de corresponder ao texto.

2.7 Remover Captura e Lixeira NOVO

A remoção individual move a captura para a lixeira após confirmação. Na lixeira, o usuário pode restaurar um item para o histórico ou esvaziá-la definitivamente.

Regras de negócio:

- Toda remoção exige confirmação explícita (`s/n`);
- Esvaziar a lixeira é a única operação que apaga dados de forma definitiva.

2.8 Ver Estatísticas AMPLIADO

Apresenta o total de capturas e de itens na lixeira; quantidade e percentual por tipo de fonte, por categoria e por origem (OCR ou manual); capturas feitas hoje, nos últimos 7 e nos últimos 30 dias; e o total de traduções salvas.

2.9 Exportar e Importar Capturas NOVO

A exportação gera o arquivo `dados/exportacao_capturas.json` com todas as capturas do histórico. A importação lê um arquivo nesse formato e adiciona as capturas que ainda não existem. Esse é o mecanismo de integração por arquivo da solução: permite levar o histórico para outra máquina e disponibilizá-lo a outras partes do sistema.

Contrato do arquivo de integração: uma lista JSON em UTF-8 com as capturas, no formato descrito na seção 4. Apenas `id` e `texto` (mínimo de 3 caracteres) são obrigatórios na importação; os demais campos (`tipo`, `categoria`, `origem`, `criado_em`, `atualizado_em` e `traducoes`) recebem valores padrão quando ausentes ou inválidos. O arquivo `exemplos/capturas_exemplo.json` traz 3 capturas prontas para importar.

Regras de negócio:

- Capturas repetidas (mesmo `id` ou mesmo texto) e itens que estão na lixeira não são importados novamente;
- Registros inválidos do arquivo são descartados e campos desconhecidos são corrigidos para valores padrão.

2.10 Limpar Histórico MODIFICADO

Move todas as capturas para a lixeira após confirmação, mostrando a quantidade de itens afetados. Diferente da Sprint 3, a operação pode ser desfeita pela lixeira.

3. APIS EXTERNAS CONSUMIDAS NOVO

Todas as APIs são consumidas com a biblioteca padrão (`urllib.request`), sem dependências externas, e são gratuitas. Apenas o OCR.space exige uma chave gratuita, informada pela variável de ambiente `OCR_SPACE_API_KEY`; a chave não fica no código-fonte. Sem a chave, o programa continua funcionando e oferece a digitação manual do texto.

API	Uso no projeto	Endpoint / Autenticação
OCR.space	Reconhecimento do texto contido em imagens (arquivo local enviado em base64 ou URL)	`POST api.ocr.space/parse/image` Chave gratuita lida da variável de ambiente `OCR_SPACE_API_KEY` (não incluída no código)
MyMemory	Tradução automática com detecção do idioma de origem	`GET api.mymemory.translated.net/get` Sem chave

API	Uso no projeto	Endpoint / Autenticação
Wikipédia	Busca de artigo e resumo do assunto capturado	`GET pt.wikipedia.org/w/api.php` e `/api/rest_v1/page/summary/` Sem chave

Todas as chamadas passam pela função `requisitar_json()`, que centraliza a montagem da requisição, o timeout de 20 segundos e a conversão de qualquer falha em uma exceção própria, `ErroAPI`, com mensagem amigável. Isso inclui falhas no meio da requisição, como conexão derrubada ou resposta cortada. Assim, cada funcionalidade trata apenas um tipo de erro e o menu continua funcionando mesmo sem internet.

4. PERSISTÊNCIA DE DADOS NOVO

Os dados são armazenados em `dados/capturas.json`, criado automaticamente na primeira execução e carregado ao iniciar o programa. O arquivo contém um dicionário com duas listas, `capturas` e `lixeira`. Cada captura é um dicionário:

```
{
  "id": "634e31b4",
  "texto": "O Pequeno Príncipe\nAntoine de Saint-Exupéry",
  "tipo": "Texto impresso",
  "categoria": "Livros",
  "origem": "ocr",
  "criado_em": "2026-10-15T11:56:10",
  "atualizado_em": null,
  "traducoes": [
    {"idioma": "en", "texto": "The Little Prince ...", "em": "2026-10-15T12:01:44"}
  ]
}
```

- **Salvamento automático:** cada funcionalidade retorna `True` quando altera os dados, e `main()` salva o arquivo nesse momento;
- **Gravação segura:** os dados são gravados em um arquivo temporário e depois renomeados, evitando arquivo corrompido se o programa for interrompido no meio da gravação;
- **Recuperação:** se o arquivo estiver corrompido, uma cópia é guardada em `capturas.bak` e o programa inicia vazio, sem travar;
- **Validação ao carregar:** registros sem os campos obrigatórios são descartados e valores desconhecidos são substituídos por padrões;
- **Aviso de registros descartados:** se algum registro for inválido, o sistema informa quantos foram ignorados e guarda o arquivo original em `capturas.bak` antes do próximo salvamento, evitando perda silenciosa de dados;
- **Interrupção:** ao pressionar `Ctrl+C`, os dados são salvos antes de encerrar.

5. TRATAMENTO DE EXCEÇÕES NOVO

Situação	Exceções tratadas	Comportamento
Entrada numérica do usuário	`ValueError`	`ler_inteiro()` informa o erro e pede novamente, validando também o intervalo
Leitura do arquivo de dados	`FileNotFoundError`, `json.JSONDecodeError`, `UnicodeDecodeError`, `OSError`	Inicia vazio; arquivo corrompido vira `.bak`; listas com formato inválido são ignoradas; registros inválidos são contados e o usuário é avisado
Gravação de arquivos	`OSError`, `TypeError`	Mensagem de erro; o programa continua
Imagem para OCR	`FileNotFoundError`, `ValueError` (extensão/tamanho), `OSError`	Mensagem clara e opção de digitar o texto
Chave do OCR ausente	`ErroAPI` (levantada por `extrair_texto_ocr`)	Aviso claro e oferta de digitação manual
Chamadas às APIs	`HTTPError`, `URLError`, `TimeoutError`, `JSONDecodeError`, `KeyError` → `ErroAPI`; conexão derrubada ou resposta cortada (`OSError`, `http.client.HTTPException`, como `ConnectionResetError`, `IncompleteRead` e `RemoteDisconnected`) → `ErroAPI`	Mensagem amigável; volta ao menu
Abrir link no navegador	`webbrowser.Error`	Mensagem de aviso; o link continua na tela para ser copiado
Importação de arquivo	`FileNotFoundError`, `JSONDecodeError`, `UnicodeDecodeError`, `OSError`	Informa o problema (inclusive a linha do erro no JSON)
Encerramento inesperado	`KeyboardInterrupt`, `EOFError`	Salva os dados e encerra com mensagem

6. CONCEITOS DE PYTHON APLICADOS

Conceito	Aplicação no Projeto
Subrotinas (funções)	47 funções com responsabilidade única, separando funções puras (`criar_captura()`, `filtrar_capturas()`, `contar_por_campo()`, `montar_url_busca_google()`) das funções de entrada/saída
Parâmetros e retorno AMPLIADO	Os dados são criados em `main()` e passados por parâmetro; parâmetros com valor padrão (`limite=45`, `permitir_vazio=False`); as funções do menu retornam `bool` indicando se houve alteração, usado para decidir quando salvar
Validação de entrada AMPLIADO	Funções reutilizáveis `ler_texto()`, `ler_inteiro()`, `escolher_opcao()` e `confirmar()`, todas com repetição até a entrada ser válida
Tratamento de exceções NOVO	`try/except` em entradas, arquivos e APIs; exceção própria `ErroAPI` encadeada com `raise ... from`
Arquivos e JSON NOVO	`json.load` / `json.dump` com `encoding="utf-8"` e `ensure_ascii=False`; `pathlib.Path` para caminhos

Conceito	Aplicação no Projeto
Consumo de APIs NOVO	`urllib.request` para GET e POST, `urllib.parse` para codificar parâmetros, `base64` para enviar a imagem
Listas e Dicionários	Capturas como lista de dicionários; dicionários de domínio (`TIPOS_FONTE`, `CATEGORIAS`, `IDIOMAS`); dicionário de ações do menu mapeando opção → função; dicionários de contagem com `dict.get(k, 0) + 1`; list comprehensions e conjuntos (`set`) para detectar duplicatas
Estruturas de decisão e repetição	`match-case` no menu (com guarda `case _ if`); `while True` nas validações; `for` / `enumerate` nas listagens
Boas práticas	Type hints e docstrings em todas as funções; constantes de configuração no topo; segredos fora do código (chave da API em variável de ambiente); prefixos `[!]` para erros e `[✓]` para sucesso; `if __name__ == "__main__"`
Testes automatizados NOVO	`test_projeto.py` com `unittest` (23 testes, sem acesso à internet): modelo de dados, persistência, conversão de falhas de conexão em `ErroAPI` (servidor local simulado) e fluxos do menu. Execução: `python -m unittest -v`

7. ESTRUTURA DO CÓDIGO-FONTE

O arquivo `projeto.py` é organizado em blocos: configuração (constantes e dicionários de domínio), entrada de dados e validações, modelo de dados (funções puras), persistência em JSON, APIs externas, funcionalidades do menu e programa principal. O projeto usa apenas a biblioteca padrão do Python 3.10+. Os testes automatizados ficam em `test_projeto.py` e o arquivo `exemplos/capturas_exemplo.json` traz capturas prontas para a importação.

Função	Responsabilidade
Entrada de dados e validações	
`ler_texto(mensagem, minimo, permitir_vazio)` NOVO	Lê texto com tamanho mínimo, opcionalmente aceitando vazio
`ler_inteiro(mensagem, minimo, maximo)` NOVO	Lê inteiro no intervalo, tratando `ValueError`
`escolher_opcao(titulo, opcoes, permitir_manter)` NOVO	Exibe um dicionário de opções e retorna o valor escolhido
`confirmar(pergunta)` NOVO	Pergunta s/n e retorna `bool`
`texto_valido(texto)` / `resumir_texto(texto, limite)`	Valida tamanho mínimo / trunca texto para listagens
Modelo de dados (funções puras)	
`criar_captura(texto, tipo, categoria, origem)`	Monta e retorna o dicionário da captura
`captura_valida(dado)` / `completar_captura(dado)` NOVO	Validam e normalizam registros lidos de arquivo
`atualizar_captura(captura, texto, tipo, categoria)` NOVO	Aplica a edição e retorna se houve alteração

Função	Responsabilidade
`filtrar_capturas(capturas, termo, categoria)` NOVO	Retorna as capturas que atendem à busca/filtro
`contar_por_campo(capturas, campo)` MODIFICADO	Generaliza a antiga `contar_capturas_por_tipo()` para qualquer campo
`contar_por_periodo(capturas, referencia)` NOVO	Conta capturas de hoje, 7 e 30 dias
`mesclar_capturas(dados, novas)` NOVO	Adiciona capturas importadas sem duplicar; retorna quantas entraram
`montar_url_busca_google(texto)` / `montar_url_traducao(texto, idioma)`	Retornam as URLs de pesquisa e tradução
Persistência NOVO	
`carregar_dados(caminho)`	Lê o JSON, com recuperação de arquivo corrompido e aviso de registros inválidos descartados
`salvar_json(caminho, conteudo)` / `salvar_dados(caminho, dados)`	Gravação segura em JSON; retornam sucesso
APIs externas NOVO	
`requisitar_json(url, metodo, formulario)`	Requisição HTTP centralizada; converte falhas (inclusive conexão interrompida) em `ErroAPI`
`resolver_caminho(caminho)`	Localiza o arquivo informado também a partir da pasta do projeto, permitindo executar o programa de qualquer diretório
`ler_imagem_base64(caminho)`	Valida extensão/tamanho e codifica a imagem
`extrair_texto_ocr(origem, idioma)`	Retorna o texto reconhecido pelo OCR.space (exige a chave configurada)
`traduzir_via_api(texto, idioma_destino)`	Retorna a tradução da API MyMemory
`buscar_resumo_wikipedia(termo)`	Retorna título, resumo e link (ou `None`)
Funcionalidades do menu (recebem `dados` e retornam `bool`)	
`capturar_texto()`, `pesquisar_no_google()`, `traduzir_texto()`, `consultar_wikipedia()`, `visualizar_historico()`, `editar_captura()`, `remover_captura()`, `gerenciar_lixeira()`, `visualizar_estatisticas()`, `exportar_capturas()`, `importar_capturas()`, `limpar_historico()`	Uma função por opção do menu (seção 2); usam as funções auxiliares acima, como `selecionar_captura(capturas, acao)`, reaproveitada por cinco funcionalidades
`oferecer_abrir_no_navegador(url)` NOVO	Pergunta se o usuário quer abrir o link no navegador e abre com `webbrowser`; usada por `pesquisar_no_google()` e `traduzir_texto()`
`main()`	Carrega os dados, executa o loop do menu e salva após cada alteração

Loop principal

```
def main() -> None:
    dados = carregar_dados(ARQUIVO_DADOS)
    acoes = {
        "1": capturar_texto,       "2": pesquisar_no_google,
        "3": traduzir_texto,       "4": consultar_wikipedia,
        "5": visualizar_historico, "6": editar_captura,
        "7": remover_captura,      "8": gerenciar_lixeira,
        "9": visualizar_estatisticas,
        "10": exportar_capturas,   "11": importar_capturas,
        "12": limpar_historico,
    }
    try:
        while True:
            exibir_menu(len(dados["capturas"]))
            opcao = input("Escolha uma opção: ").strip()
            match opcao:
                case "0":
                    print("\nEncerrando o sistema. Até logo!")
                    break
                case _ if opcao in acoes:
                    if acoes[opcao](dados) and salvar_dados(ARQUIVO_DADOS, dados):
                        print("    (alterações salvas)")
                case _:
                    print("\n[!] Opção inválida. Escolha um número do menu.")
    except (KeyboardInterrupt, EOFError):
        salvar_dados(ARQUIVO_DADOS, dados)
```

Fluxo de execução

1. O programa inicia por `main()`, chamada via `if __name__ == "__main__"`.
2. Os dados salvos em `dados/capturas.json` são carregados (ou uma estrutura vazia é criada).
3. O menu é exibido em loop até que o usuário selecione a opção 0 (Sair).
4. A opção escolhida é localizada no dicionário `acoes` e a função correspondente é chamada com `dados` como parâmetro.
5. Se a função retornar `True` (dados alterados), o arquivo JSON é salvo; em seguida, o programa volta ao menu.

8. JUSTIFICATIVA DE ALTERAÇÕES EM RELAÇÃO À SPRINT 3 NOVO

Esta seção documenta, conforme exigido pelo Sprint 3 e 4 Challenge, o que foi removido, modificado e adicionado no código em relação à versão entregue na Sprint 3, e como cada ponto do feedback recebido foi tratado.

8.1 O que foi removido

- O histórico mantido apenas em memória (lista criada vazia a cada execução), substituído por dados persistidos em JSON.
- A captura "simulada", em que o usuário digitava o texto que a câmera supostamente detectaria; a digitação permanece apenas como alternativa quando o OCR não é possível.
- A função `indice_valido()` baseada em `isdigit()`, substituída por `ler_inteiro()` com `try/except ValueError`.

- As funções `ler_tipo_fonte()` e `ler_idioma_destino()`, substituídas pela função genérica `escolher_opcao()`, reutilizada para tipo de fonte, categoria e idiomas.

8.2 O que foi modificado

- **Captura de texto:** `simular_captura_ocr()` tornou-se `capturar_texto()`, com OCR real via API.
- **Tradução:** deixou de apenas gerar um link; agora o texto é traduzido pela API MyMemory e a tradução fica salva na captura (o link do Google Tradutor foi mantido como complemento). Foi adicionado o Português como idioma de destino.
- **Pesquisa no Google:** além de gerar o link, passou a oferecer a abertura direta no navegador.
- **Estrutura da captura:** o dicionário passou de `{texto, tipo}` para `{id, texto, tipo, categoria, origem, criado_em, atualizado_em, traducoes}`.
- **Estado do programa:** a lista `historico` deu lugar ao dicionário `dados`, com as listas `capturas` e `lixeira`; as funções do menu passaram a retornar `bool` indicando alteração.
- **Menu:** o `match-case` passou a usar um dicionário de ações (opção → função), o que simplificou a inclusão das novas opções; o menu foi renumerado de 6 para 12 funcionalidades.
- **Limpar histórico:** passou a mover as capturas para a lixeira, permitindo desfazer a operação.
- **Estatísticas:** `contar_capturas_por_tipo()` foi generalizada em `contar_por_campo()`, permitindo estatísticas por tipo, categoria e origem.
- **README:** o título ainda indicava "sprint-2-python"; foi reescrito com instruções de execução, funcionalidades, APIs e persistência.

8.3 O que foi adicionado

- Persistência em JSON com salvamento automático, gravação segura e recuperação de arquivo corrompido (seção 4).
- Consumo de três APIs externas: OCR.space, MyMemory e Wikipédia (seção 3).
- Tratamento de exceções com `try/except` em entradas, arquivos e APIs, e a exceção própria `ErroAPI` (seção 5).
- Novas funcionalidades: Saiba mais na Wikipédia, editar captura, remover captura individual, lixeira com restauração, busca e filtro no histórico, exportação e importação em JSON.
- Categorias para as capturas e estatísticas por categoria, origem e período.
- Imagens de exemplo na pasta `exemplos/` para testar o OCR.
- Tratamento de falhas no meio da requisição (conexão derrubada ou resposta cortada), convertidas em `ErroAPI`.
- Chave do OCR.space fora do código-fonte (variável de ambiente `OCR_SPACE_API_KEY`), com digitação manual quando ela não está configurada.
- Aviso e backup (`capturas.bak`) quando registros inválidos são descartados ao carregar os dados.
- Confirmação do termo de busca da Wikipédia e opção de abrir links no navegador.
- Contrato documentado do arquivo JSON de integração e arquivo de exemplo importável (`exemplos/capturas_exemplo.json`).
- Testes automatizados com `unittest` (`test_projeto.py`).

8.4 Atendimento ao feedback da Sprint 3

Ponto do feedback	Como foi tratado na Sprint 4
"O 'OCR' é o usuário digitando o texto"	OCR real de imagens pela API OCR.space (seção 2.1)
"Escopo modesto (não remove item individual, não edita)"	Edição de captura, remoção individual com lixeira e restauração, busca/filtro, Wikipédia, exportação/importação e estatísticas ampliadas (seções 2.4 a 2.9)
"Sem persistência"	Dados salvos em `dados/capturas.json` após cada alteração (seção 4)
"Sem try/except"	Tratamento de exceções em todo o programa (seção 5)
"O README ainda diz 'sprint-2-python'"	README reescrito para a Sprint 4

Os pontos elogiados na Sprint 3 foram preservados: type hints e docstrings em todas as funções, separação entre funções puras e de entrada/saída, reaproveitamento de `selecionar_captura()` com a ação recebida por parâmetro, contagem com `dict.get(k, 0) + 1`, `match-case` no menu, estado local em `main()` e confirmação explícita antes de operações destrutivas.